

gmpfrxx_mkII

A header-only C++17 expression-template wrapper for GMP, MPFR, and GNU MPC

Version 1.4.1

Release tag v1.4.1

NAKATA Maho¹

¹RIKEN Cluster for Pioneering Research, 2-1 Hirosawa, Wako-City, Saitama 351-0198, JAPAN

August 27, 2026

Abstract

gmpfrxx_mkII provides value-semantic C++ classes for arbitrary-precision integers, rationals, real floating-point numbers, and complex numbers, layered on the GNU multiple-precision libraries GMP, MPFR, and GNU MPC. An expression-template arithmetic layer removes most temporaries from compound expressions. The wrapper is designed as numerical infrastructure: each floating wrapper has exactly the size and alignment of its backend C object, the default-state policy is explicit and predictable across threads and shared libraries, and a small set of opt-in compile-time fast paths is available for BLAS-like kernels. This manual documents the public API, the precision and default-state model, the expression-template lifetime contract, and the behavioral guarantees that callers can rely on.

Contents

1	Introduction	2
1.1	A from-scratch implementation	2
1.2	Design priorities	3
1.3	Public and internal namespaces	3
2	Installation and requirements	3
2.1	Toolchain	3
2.2	The MPFR TLS requirement	3
2.3	Compiling a program	3
3	Headers and namespaces	4
4	Public types overview	4
5	Quick start	5
6	Precision and default state	5
6.1	Initial defaults	5
6.2	GMP MPF default precision	5
6.2.1	Default-context modes	5
6.3	MPFR default state	6
6.3.1	Per-thread first-use initialization	6
6.4	MPC default state	6
6.5	Environment variables	7

7	Expression templates and the lifetime contract	7
7.1	Generative random expressions	7
8	Scalar policy and common types	8
8.1	Explicit fixed-binary import adapters	8
9	Type reference	8
9.1	<code>gmpxx::mpz_class</code>	8
9.2	<code>gmpxx::mpq_class</code>	9
9.3	<code>gmpxx::mpf_class</code>	9
9.4	<code>gmpxx::mpfc_class</code>	10
9.5	<code>mpfrxx::mpfr_class</code>	10
9.6	<code>mpfrxx::mpc_class</code>	11
10	Rounding and evaluation contexts	12
10.1	MPFR rounding	12
10.2	MPC contexts	12
11	Compile-time fast paths	12
12	<code>mpq_class</code> stream input	13
13	Move semantics and layout guarantees	13
14	Platform notes	14
15	Benchmark results	14
16	Version information	15
17	License	15

1 Introduction

`gmpfrxx_mkII` is a header-only C++17 wrapper library for GMP, MPFR, and GNU MPC. In its default mode it requires no separate compiled library: including a header and linking the relevant backend C libraries is sufficient. It exposes four numeric domains.

- GMP-backed exact integers (`mpz_class`) and rationals (`mpq_class`).
- GMP-backed binary floating point (`mpf_class`) and an MPF-backed complex type (`mpfc_class`) built from two `mpf_class` components.
- MPFR-backed correctly-rounded real floating point (`mpfr_class`).
- GNU MPC-backed complex floating point (`mpc_class`).

1.1 A from-scratch implementation

This is a from-scratch implementation built directly on the GMP, MPFR, and MPC C APIs. It does not include `gmpxx.h`, does not link `libgmpxx`, and does not bridge to or wrap any existing C++ wrapper. The expression-template layer, the class designs, and the default-state machinery are all original to this project. The result is compatibility-oriented in spirit — familiar type names and operator behavior — but it is not a full `gmpxx.h` clone, and it deliberately diverges where a clearer or more predictable contract is worthwhile.

1.2 Design priorities

The library favors predictability over implicit convenience. The default precision is a fixed, documented value; assignment to an existing object never silently changes its precision; a single scalar constructor argument is always a *value*, never a precision selector; and the floating wrappers add no per-object metadata around the backend C object, so arrays of wrappers keep native stride. These choices target use as numerical infrastructure — for example dense linear-algebra kernels — rather than ad hoc scripting convenience.

1.3 Public and internal namespaces

The public API lives in two namespaces:

```
namespace gmpxx;    // GMP-only API
namespace mpfrxx;   // MPFR API, plus the MPC API when the MPC header is included
```

The implementation lives in an internal namespace that is not part of the public API:

```
namespace gmpfrxx_mkII::detail;
```

In particular, `namespace gmpfrxx` is *not* a public API namespace. The `gmpfrxx_mkII` name appears only as the internal `detail` namespace and as header and macro prefixes.

This manual assumes familiarity with the underlying C libraries. It describes what the wrapper guarantees, not the internals of GMP, MPFR, or MPC.

2 Installation and requirements

2.1 Toolchain

Component	Requirement
Compiler	A conforming C++17 compiler (GCC, Clang, or MSVC).
GMP	Required by every public header.
MPFR	Required by <code>mpfrxx_mkII.h</code> and the aggregator. Must be built with thread-local storage.
GNU MPC	Required only for <code>mpfrxx::mpc_class</code> .

2.2 The MPFR TLS requirement

The MPFR default precision, rounding mode, and exponent range are treated as the single source of truth and are read from `libmpfr`'s thread-local state. The project therefore requires an MPFR build with thread-local storage: a CMake configuration must verify that `mpfr_buildopt_tls_p()` exists and reports TLS enabled, and must fail otherwise. There is no fallback default-state implementation for a non-TLS MPFR build.

2.3 Compiling a program

```
#include <gmpfrxx_mkII.h>
#include <iostream>

int main() {
    mpfrxx::mpfr_class x = 2;
    mpfrxx::mpfr_class y = sqrt(x);    // 512-bit by default
    std::cout << y << '\n';
}
```

```
c++ -std=c++17 example.cpp -lmpc -lmpfr -lgmp
```

If only real MPFR arithmetic is needed, include `mpfrxx_mkII.h` and link `-lmpfr -lgmp`; GNU MPC is not required.

3 Headers and namespaces

Headers are split by required backend and link dependency so that a translation unit pays only for the backends it uses.

Header	Public API	Link dependency
<code>gmpxx_mkII.h</code>	<code>gmpxx::mpz_class</code> , <code>mpq_class</code> , <code>mpf_class</code> , <code>mpfc_class</code>	<code>-lgmp</code>
<code>mpfrxx_mkII.h</code>	<code>mpfrxx::mpfr_class</code> (+ <code>mpz_class</code> , <code>mpq_class</code> aliases)	<code>-lmpfr -lgmp</code>
<code>mpcxx_mkII.h</code>	<code>mpfrxx::mpc_class</code>	<code>-lmpc -lmpfr -lgmp</code>
<code>gmpfrxx_mkII.h</code>	aggregator of all of the above	<code>-lmpc -lmpfr -lgmp</code>

`gmpxx_mkII.h` is deliberately GMP-only. It does not include `<mpfr.h>` or `<mpc.h>`, references no `mpfr_*` or `mpc_*` symbol, and requires neither library at link time. The GMP complex type `gmpxx::mpfc_class` is implemented from two `mpf_class` components and stays in the GMP-only header rather than being split into a separate complex header.

`mpfrxx_mkII.h` requires GMP and MPFR but not GNU MPC. `mpcxx_mkII.h` includes `mpfrxx_mkII.h` automatically and layers the complex type on top.

Two namespaces are used. `gmpxx` holds the GMP-only types. `mpfrxx` holds the MPFR and MPC types and re-exports the GMP integer and rational types as aliases:

```
namespace mpfrxx {
    using ::gmpxx::mpz_class;
    using ::gmpxx::mpq_class;
}
```

4 Public types overview

Type	Backend	Notes
<code>gmpxx::mpz_class</code>	GMP <code>mpz_t</code>	Exact integer.
<code>gmpxx::mpq_class</code>	GMP <code>mpq_t</code>	Exact rational.
<code>gmpxx::mpf_class</code>	GMP <code>mpf_t</code>	Binary float; no IEEE NaN/Inf/signed-zero.
<code>gmpxx::mpfc_class</code>	two <code>mpf_class</code>	GMP complex; no GNU MPC dependency.
<code>mpfrxx::mpfr_class</code>	MPFR <code>mpfr_t</code>	Correctly-rounded real float.
<code>mpfrxx::mpc_class</code>	GNU MPC <code>mpc_t</code>	Complex float.

The two complex families are distinct by design. `gmpxx::mpfc_class` is MPF-backed; `mpfrxx::mpc_class` is MPC-backed. Mixed arithmetic between the families is intentionally rejected at compile time.

Because `mpf_class` and `mpfc_class` are GMP `mpf_t`-backed, they do not model IEEE signed zero, NaN, or infinity, and cannot reproduce signed-zero branch cuts. In particular `gmpxx::atan2(0,0)` returns `+0` and the four IEEE signed-zero cases are indistinguishable. Use `mpfr_class` or `mpc_class` when signed-zero branch behavior is part of the numeric contract.

5 Quick start

```
#include <gmpfrxx_mkII.h>
using namespace gmpxx;
using namespace mpfrxx;

mpz_class n = 1;
for (int k = 1; k <= 50; ++k) n *= k;           // 50!

mpq_class q("22/7");
q.canonicalize();                             // arithmetic-ready

mpf_class a(1.0, 256), b = sqrt(mpf_class(2, 256));
mpf_class c = a + b * b;                       // evaluated immediately

mpfr_class x = const_pi();                     // 512-bit pi
mpfr_class y = sin(x / 6);

mpc_class z(mpfr_class(0), mpfr_class(1));     // i
mpc_class w = exp(z * const_pi());             // ~ -1
```

6 Precision and default state

6.1 Initial defaults

The wrapper default floating precision is **512 bits** for MPF, MPFR, and MPC, unless overridden by environment variables (Section 6.5).

Assignment to an *existing* floating object preserves the left-hand-side precision in every domain. Changing a wrapper default affects only *subsequently constructed* objects.

6.2 GMP MPF default precision

GMP's `mpf_set_default_prec()` / `mpf_get_default_prec()` are process-global, not thread-local. `mpf_class` therefore does not use GMP's global default as its policy; it owns its own default. Default construction uses `mpf_init2(..., gmpxx::default_mpf_precision_bits())`.

Two explicit escape hatches expose the GMP global C state for code that intentionally wants it; these are *not* the wrapper default policy:

```
gmpxx::unsafe_get_gmp_global_mpf_precision_bits();
gmpxx::unsafe_set_gmp_global_mpf_precision_bits(bits);
```

6.2.1 Default-context modes

`GMPXX_MKII_DEFAULT_CONTEXT_MODE` is a program-wide build setting. All translation units linked into one program must use the same value.

Frozen-env mode (default). Header-only; no provider object file; no header-owned mutable TLS. The default MPF precision is 512 bits unless overridden by `GMPXX_DEFAULT_MPF_PRECISION_BITS`, and the environment is read once and then frozen. Runtime mutation of the wrapper default is unsupported: `gmpxx::set_default_mpf_precision_bits(...)` is a compatibility no-op, and `gmpxx::reload_default_mpf_precision_bits_from_environment()` keeps the frozen value. Because the frozen value lives in a function-local static, each linked image may observe its own frozen default; this is acceptable for the simple header-only mode but is not a cross-DSO mechanism.

External-provider mode. Opt-in and not header-only: exactly one linked provider implementation owns the mutable per-thread default-context storage. Missing provider symbols are a link error. In this mode `gmpxx::set_default_mpf_precision_bits(...)` sets the calling thread's provider-owned default and `gmpxx::default_mpf_precision_guard` restores the previous value on scope exit. Do not embed the provider object in multiple shared libraries; designate a single owning library. Provider identity is checkable through `gmpxx_mkII_default_context_provider_token_v1()`, and all participating DSOs must observe the same token.

6.3 MPFR default state

MPFR default precision, rounding mode, and exponent range are owned by `libmpfr`'s thread-local state; the wrapper keeps no separate mutable storage for them. The public MPFR default API routes directly through MPFR:

```
mpfrxx::default_precision_bits();           // mpfr_get_default_prec()
mpfrxx::set_default_precision_bits(bits);   // mpfr_set_default_prec()
mpfrxx::default_rounding_mode();
mpfrxx::set_default_rounding_mode(rnd);
mpfrxx::default_emin(); mpfrxx::default_emax();
mpfrxx::set_default_exponent_range(emin, emax);
```

6.3.1 Per-thread first-use initialization

On a thread's first wrapper default access, if the MPFR default state still has its library-initial values (precision = 53, rounding = `MPFR_RNDN`, `emin/emax` at MPFR's defaults), the wrapper applies its 512-bit policy together with any `MPFRXX_*` environment overrides. If application code or another linked image has already changed any MPFR default field in that thread, the wrapper leaves the existing state alone. A DSO-local one-shot flag avoids repeatedly probing the defaults from hot paths; it stores no precision or rounding value of its own.

To establish an explicit boundary — for example before applying raw `mpfr_set_default_prec(...)` — call `mpfrxx::initialize_thread_defaults()` first. After the one-shot initialization has run, later raw MPFR default-state changes in the same thread are not overwritten by implicit wrapper access.

Default setters affect only the calling thread, and only subsequently constructed objects. `mpfrxx::reload_mpfr_defaults_from_environment()` reloads environment-derived defaults for the calling thread only. Applications that want consistent runtime defaults across worker threads must call a setter or reload API inside each thread.

6.4 MPC default state

MPC inherits the MPFR thread-local defaults: without an MPC-specific override, default real and imaginary precision equal the MPFR default precision, and default real and imaginary rounding equal the MPFR default rounding.

MPC-specific overrides affect *only* `mpc_class` defaults; they do not touch MPFR defaults. Precision and rounding may be symmetric or asymmetric:

```
mpfrxx::set_default_mpc_precision_bits(prec);           // symmetric
mpfrxx::set_default_mpc_precision_bits(real, imag);     // asymmetric
mpfrxx::set_default_mpc_rounding_mode(rnd);             // symmetric
mpfrxx::set_default_mpc_rounding_mode(real_rnd, imag_rnd); // asymmetric
```

Assigning a real value (an `mpfr_class`, `mpz_class`, `mpq_class`, or a supported scalar) to an `mpc_class` sets the imaginary component to +0, following GNU MPC's `mpc_set_fr`-family semantics.

6.5 Environment variables

```
GMPXX_DEFAULT_MPF_PRECISION_BITS

MPFRXX_DEFAULT_PRECISION_BITS
MPFRXX_DEFAULT_EMIN
MPFRXX_DEFAULT_EMAX
MPFRXX_DEFAULT_ROUNDING_MODE      # RNDN RNDZ RNDU RNDD RNDA RNDF

MPCXX_DEFAULT_PRECISION_BITS
MPCXX_DEFAULT_REAL_PRECISION_BITS
MPCXX_DEFAULT_IMAG_PRECISION_BITS
MPCXX_DEFAULT_ROUNDING_MODE
MPCXX_DEFAULT_REAL_ROUNDING_MODE
MPCXX_DEFAULT_IMAG_ROUNDING_MODE
```

Environment variables are process-startup configuration. Finalize all `GMPXX_*`, `MPFRXX_*`, and `MPCXX_*` variables before first wrapper default access and before worker threads begin numeric work. Concurrent `setenv/unsetenv/putenv` racing with wrapper default reads is outside the supported contract.

Invalid *user* environment values do not terminate the process: they are ignored with a diagnostic on `stderr` and the built-in default is used. By contrast, an external default-context provider with a mismatched ABI version or an internally inconsistent record may terminate the process after a diagnostic, because that indicates a binary-integration error rather than a configuration typo.

The `MPCXX_*` component variables override only their respective component; the other component inherits the corresponding MPC default or, failing that, the MPFR default. Asymmetric MPC environment precision and rounding are therefore valid.

7 Expression templates and the lifetime contract

Public arithmetic operators (+, -, *, /, unary -) and compound assignments return expression nodes rather than evaluated values. A node may **borrow** an lvalue wrapper operand by raw reference and may **own** an rvalue operand.

This is a user-visible contract: **do not bind an expression to `auto` and let it outlive its operands.**

```
// BAD: e holds raw references to a and b.
auto e = a + b;
mutate_or_destroy(a);
gmpxx::mpf_class r = e;           // dangling if a was destroyed,
                                   // or stale if a was mutated.

// GOOD: evaluate immediately.
gmpxx::mpf_class r = a + b;

// GOOD: explicit snapshot when a stable value is needed.
auto snapshot = gmpxx::mpf_class(a + b);
```

Expression node types are `[[nodiscard]]`, so a discarded standalone expression such as `a + b`; produces a compiler diagnostic. This does not make a saved node lifetime-safe.

7.1 Generative random expressions

Random getters follow the same policy as upstream `gmpxx.h`: they return generator expression nodes, not value snapshots. This includes `get_z_bits`, `get_z_range`, `get_f` on `gmpxx::`

`gmp_randclass`, the matching getters on `mpfrxx::mpfr_randclass`, and the MPFR floating getters such as `get_fr(...)`. Evaluating such a node consumes the associated `gmp_randstate_t` and may produce a fresh value.

The number of times a random node embedded in an expression tree is generated is *not* a semantic contract. If one draw must be reused, materialize it first:

```
mpfrxx::mpfr_class r = state.get_fr(prec);
mpfrxx::mpfr_class y = r + r;           // reuses one draw
```

Random-state generation is not internally synchronized. Use a state from one thread at a time, or protect it with an external mutex.

8 Scalar policy and common types

Supported real scalar leaves are non-`bool` integral types up to 64 bits, `float`, and `double`. Integral leaves are normalized to signed or unsigned 64-bit storage before evaluation, so `long` and `unsigned long` are accepted through that normalization rather than through GMP's native `long` ABI. Supported scalars promote with `mpfr_class` to `mpfr_class` and with `mpc_class` to `mpc_class`.

`bool`, `wchar_t`, `char16_t`, `char32_t`, `long double`, and `__int128` are intentionally outside the scalar-leaf contract and must not become common-type promotion backdoors. (Direct `mpz_class` construction from `__int128` is supported where the compiler provides it, but `__int128` is still rejected as an expression scalar leaf.)

8.1 Explicit fixed-binary import adapters

Binary80 and binary128 name numeric formats, not stable C++ native types. The adapter layer therefore uses semantic wrappers parameterized by the consumer-selected native type:

```
auto r80 = gmpfrxx_mkII::adapters::make_binary80_source(value80);
auto r128 = gmpfrxx_mkII::adapters::make_binary128_source(value128);
mpfrxx::mpfr_class x80(r80);
mpfrxx::mpfr_class x128(r128);
```

The corresponding `make_binary80_complex_source(real, imag)` and `make_binary128_complex_source(real, imag)` wrappers import components without depending on extended `std::complex` support. Supported native types are selected by capability traits: `binary80` accepts a verified `long double` or `_Float64x` representation; `binary128` accepts `_Float128`, `__float128`, or a verified `binary128 long double`. Generic `long double` scalar promotion remains disabled, so the semantic wrapper is required.

When `mpcxx_mkII.h` or `gmpfrxx_mkII.h` is included, `std::complex<double>` is accepted as an MPC scalar operand. It promotes with `mpc_class` to `mpc_class`, and with `mpfr_class` to `mpc_class` because the result has a complex component. This support is MPC-only and does not make `mpfrxx_mkII.h` depend on GNU MPC.

9 Type reference

This section lists the principal public surface of each type. Constructors, accessors, and free functions follow GMP/MPFR/MPC naming conventions where a direct correspondence exists.

9.1 `gmpxx::mpz_class`

Construction. Default; copy/move; from `const char*` or `std::string` with an optional base (base 0 auto-detects); explicit from `double` (rejecting non-finite input) and from `mpz_srcptr`

; implicit from supported integral scalars (and from `__int128` where available); explicit from `mpf_class` and `mpq_class`; from integer/rational expression nodes. Construction from `bool` is deleted.

Accessors and queries.

```
std::string get_str(int base = 10) const;    // also to_string(base)
int set_str(const char* s, int base = 10);
double get_d() const;
signed long get_si() const; unsigned long get_ui() const;
std::int64_t get_i64() const; std::uint64_t get_u64() const;
explicit operator bool() const noexcept;
bool fits_sint_p() const; bool fits_slong_p() const; // etc.
const mpz_t& mpz_data() const noexcept;    // also get_mpz_t()
void swap(mpz_class&) noexcept;
```

Number theory. `abs`, `sqrt` (integer square root), `gcd`, `lcm`, `factorial`, `primorial`, `fibonacci`; `operator%` for integer remainder; pre/post increment and decrement; the `_mpz` literal suffix.

9.2 `gmpxx::mpq_class`

Construction. Default; copy/move; from string with optional base; from integer and rational expression nodes; pre/post increment and decrement; the `_mpq` literal suffix.

Canonicalization. See Section 12. The arithmetic boundary is enforced through:

```
void mpq_class::canonicalize();              // throws std::domain_error on 0 denom
bool mpq_class::has_zero_denominator() const;
gmpxx::mpq_canonicalize_checked(mpq_ptr);    // raw, throws on 0 denom
gmpxx::mpq_has_zero_denominator(mpq_srcptr);
```

9.3 `gmpxx::mpf_class`

Construction.

```
mpf_class();                                // default precision
mpf_class(double v);
mpf_class(double v, mp_bitcnt_t prec);
mpf_class(T integral_scalar [, mp_bitcnt_t prec]);
mpf_class(const mpz_class& [, mp_bitcnt_t prec]);
mpf_class(const mpq_class& [, mp_bitcnt_t prec]);
mpf_class(const char* s [, mp_bitcnt_t prec], int base = 10);
mpf_class(const std::string& s [, mp_bitcnt_t prec], int base = 10);
explicit mpf_class(mpf_srcptr v [, mp_bitcnt_t prec]);
mpf_class(const mpf_class& other, mp_bitcnt_t prec);
static mpf_class with_precision(mp_bitcnt_t prec);
static mpf_class with_precision(mp_bitcnt_t prec, double v);
```

Construction from `bool` is deleted.

Accessors.

```
mp_bitcnt_t precision() const noexcept;
void set_prec(mp_bitcnt_t prec);
double get_d() const;
signed long get_si() const; unsigned long get_ui() const;
const mpf_t& mpf_data() const noexcept;
void swap(mpf_class&) noexcept;
```

Math (gmpxx namespace). A self-contained MPF real-math set: `sqrt`, `sqr`, `abs`, `neg`, `ceil`, `floor`, `trunc`, `sgn`, `hypot`, `exp`, `log`, `log1p`, `expm1`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `pow`, `gamma`, `reciprocal_gamma`, remainder helpers, and the constants `const_pi`, `const_e`, `pi_over_two`, `pi_over_four`, `two_pi`, `log_two`, `log_ten`, `inv_log_two` (each available at the default precision or at an explicitly requested precision). Most functions also accept expression-node arguments. The `_mpf` literal suffix is provided.

9.4 gmpxx::mpfc_class

A GMP-only complex type built from two `mpf_class` components.

```
mpfc_class();
mpfc_class(const mpf_class& re, const mpf_class& im);
explicit mpfc_class(const mpf_class& re);           // im = 0
explicit mpfc_class(Scalar re);
mpfc_class(RealScalar re, ImagScalar im);
static mpfc_class with_precision(mp_bitcnt_t prec);
static mpfc_class with_precision(mp_bitcnt_t re_prec, mp_bitcnt_t im_prec);

const mpf_class& real() const noexcept;
const mpf_class& imag() const noexcept;
void real(const mpf_class&); void imag(const mpf_class&);
mp_bitcnt_t precision() const noexcept;
```

Complex math (`conj`, `real`, `imag`, `norm`, `abs`, `arg`, `polar`, `sqrt`, `exp`, `log`, the trigonometric and hyperbolic functions and their inverses, integer and floating `pow`) is provided in the `gmpxx` namespace. Because the backend is `mpf_t`, IEEE signed-zero branch cuts are not modeled.

9.5 mpfrxx::mpfr_class

Construction.

```
mpfr_class();                               // default precision
mpfr_class(double v [, mpfr_prec_t prec]);
mpfr_class(T integral_scalar [, mpfr_prec_t prec]);
mpfr_class(const gmpxx::mpz_class& [, mpfr_prec_t prec]);
mpfr_class(const gmpxx::mpq_class& [, mpfr_prec_t prec]);
explicit mpfr_class(const char* / const std::string&);
mpfr_class(const char* s, mpfr_prec_t prec, int base = 0);
explicit mpfr_class(mpfr_srcptr v [, mpfr_prec_t prec]);
mpfr_class(const mpfr_class& other, mpfr_prec_t prec);
```

Construction from `bool` is deleted.

Accessors and queries.

```
mpfr_prec_t precision() const noexcept;
void set_prec(mpfr_prec_t prec);
double get_d() const;
std::string get_str() const;
std::string get_str(mpfr_exp_t& exp, int base = 10, std::size_t ndig = 0) const;
std::string to_string(std::size_t ndig = 0) const;
const mpfr_t& mpfr_data() const noexcept; // also get_mpfr_t()
void swap(mpfr_class&) noexcept;
static mpfr_prec_t default_precision() noexcept;
```

Eager math (mpfrxx namespace). A broad correctly-rounded set, including the elementary functions (`sqrt`, `cbirt`, `exp`, `exp2`, `exp10`, `expm1`, `log`, `log2`, `log10`, `log1p`, the trigonometric,

hyperbolic, and unit/ π -scaled trigonometric families and their inverses, `sec/csc/cot` and hyperbolic counterparts), special functions (`erf`, `erfc`, `gamma`, `lngamma`, `digamma`, `zeta`, `eint`, `li2`, the Bessel functions `j0/j1/ jn`, `y0/y1/yn`, `ai`), binary operations (`pow`, `atan2`, `hypot`, `fmod`, `agm`, `min`, `max`, `gamma_inc`, `beta`), fused operations (`fma`, `fmma`, `fmms`), scaling by powers of two (`mul_2ui`, `div_2ui`, `mul_2si`, `div_2si`), rounding to integers (`ceil`, `floor`, `trunc`), `setsign`, and the constants `const_pi`, `const_log2`, `const_e`, `const_log10`, `pi_over_two`, `const_euler`, `const_catalan`.

Result precision of eager math. For a multi-operand eager math function, the result precision is chosen from the MPFR object or MPFR-expression operands *before* scalar and exact GMP operands are materialized; those exact operands are then converted at the chosen precision and do not implicitly promote the result to the current default precision. If no MPFR operand contributes a positive precision, the wrapper default precision is used.

Version-dependent functions. Some MPFR functions are backend-version features (for example `powr`, the unit-based trigonometric functions, `rootn_si`, `compound_si`, `log2p1`, `exp2m1`, and `intmax` power helpers). The corresponding wrappers are exposed only when the included `<mpfr.h>` advertises the entry point; the rest of the MPFR wrapper remains available with older MPFR.

Comparisons. `mpfrxx::cmp(...)` is a total-order-style helper for ordered values; if either operand is NaN it throws `std::domain_error`. The equality, inequality, and ordering operators are NaN-aware predicates instead: equality is false for unordered operands, inequality is true, and ordering predicates are false when unordered.

9.6 `mpfrxx::mpc_class`

Construction.

```
mpc_class();
mpc_class(const mpfr_class& re, const mpfr_class& im);
explicit mpc_class(const mpfr_class& re);           // im = +0
explicit mpc_class(const gmpxx::mpz_class& re);
explicit mpc_class(const gmpxx::mpq_class& re);
explicit mpc_class(Scalar re);
explicit mpc_class(const std::complex<double>& v);
mpc_class(RealScalar re, ImagScalar im);
explicit mpc_class(const char* / const std::string&);
mpc_class(const char* s, mpfr_prec_t prec, int base = 0);
mpc_class(const char* s, mpfr_prec_t re_prec, mpfr_prec_t im_prec, int base);
```

Accessors.

```
mpfr_class real() const;      mpfr_class imag() const;
mpfr_prec_t real_precision() const noexcept;
mpfr_prec_t imag_precision() const noexcept;
mpfr_prec_t precision() const noexcept; // = real_precision()
std::string get_str(int base = 10, std::size_t ndig = 0) const;
std::string to_string(std::size_t ndig = 0) const;
const mpc_t& mpc_data() const noexcept;
```

Complex math (`mpfrxx` namespace). `sqrt`, `exp`, `log`, the trigonometric and hyperbolic families and their inverses, `pow`, `conj`, `proj`, and the projections `real`, `imag`, `abs`, `norm`, `arg`. Expression-node arguments are accepted.

Comparisons. `mpc_class` supports equality and inequality only; ordering operators are not defined. A value with a NaN component compares unequal to itself.

10 Rounding and evaluation contexts

10.1 MPFR rounding

`mpfrxx::with_rounding(value, rnd)` returns a lightweight non-owning target-bound handle that applies `rnd` to assignment and compound assignment through that handle. It carries no precision field: the destination `mpfr_class` determines the result precision exactly as a direct `mpfr_*` call would. The handle must not outlive, be stored beyond, or be used after destruction or move of the referenced object. It does not create a dynamic rounding scope and does not affect independently materialized eager math such as `sin(x)` or `const_pi(prec)`.

`mpfrxx::rounding_mode_scope` changes the calling thread’s MPFR default rounding mode for the lifetime of the scope, restores the previous value on destruction, and updates the stable-rounding cache. Nested scopes are supported. Eager MPFR math functions and constants observe the current default rounding mode and therefore observe a `rounding_mode_scope`, but they are not affected by target-bound `with_rounding` handles.

10.2 MPC contexts

MPC keeps a target-bound precision-and-rounding context because it needs separate real and imaginary precision. `mpfrxx::mpc_evaluation_context` stores real precision, imaginary precision, and an `mpc_rnd_t`; `mpfrxx::with_context(value, context)` returns a non-owning handle. Assignment and compound assignment through the handle evaluate with the cached rounding and target component precisions, avoiding per-operation default lookups in fixed-context loops. The handle may also be built from separate real and imaginary MPFR rounding modes, packed with `MPC_RND(real, imag)`. The handle must not outlive the referenced object; normal builds check that the context precision matches the target.

11 Compile-time fast paths

The three fast-path macros are compile-time contracts for hot kernels, enabled deliberately per translation unit or target. They are independent: fixed precision does not imply stable rounding, stable rounding does not imply fused arithmetic, and fused arithmetic implies no fixed-precision contract.

Macro	Meaning
<code>GMPFRXX_MKII_FAST_FIXED_PREC</code>	Applies to <code>mpf_class</code> , <code>mpfr_class</code> , <code>mpc_class</code> . Caller guarantees that all participating floating objects in the hot path share one fixed precision; a mismatch is a contract violation (asserted in debug builds). Enables swap-based move assignment and bounded scratch reuse that skip precision-restoration work.
<code>GMPFRXX_MKII_FAST_STABLE_RND</code>	Applies to MPFR (and MPC paths inheriting MPFR rounding). Caller guarantees the default rounding mode is stable during the kernel, except through wrapper setters or <code>rounding_mode_scope</code> . Replaces repeated default-rounding reads with a cached thread-local value.

Macro	Meaning
GMPFRXX_MKII_ENABLE_FMA	Applies to MPFR expression and compound-assignment paths. Maps $a += b*c$ to <code>mpfr_fma</code> , $a -= b*c$ to <code>mpfr_fma</code> with an exactly negated multiplicand, $a*b + c*d$ to <code>mpfr_fmms</code> , and $a*b - c*d$ to <code>mpfr_fmms</code> .

Numeric-semantics note. GMPFRXX_MKII_ENABLE_FMA *changes* results: fused evaluation rounds once where a materialized product-then-add rounds twice. This is why it is separate from the fixed-precision macro, which does not change arithmetic results. GMP MPF has no wrapper FMA: $a += b*c$ on `mpf_class` may use a direct multiply-then-add specialization but still follows MPF’s two-step rounding. Under GMPFRXX_MKII_FAST_FIXED_PREC, the MPF leaf-product compound path may reuse a bounded header-local thread scratch object for the intermediate product; this is a performance cache only and does not change MPF precision semantics. If a scratch pool is exhausted by a deep expression tree, the implementation falls back to a scoped backend temporary without changing observable results, precision policy, or exception behavior.

12 `mpq_class` stream input

Stream extraction for `mpq_class` and raw `mpq_ptr` intentionally matches upstream `gmpxx.h`. It uses `mpq_set_str`-compatible semantics, preserves the parsed numerator and denominator, and does *not* call `mpq_canonicalize`. Input such as 2/4 remains 2/4 at the stream layer, and structurally invalid raw rationals such as 1/0 or 0/0 are accepted by extraction rather than setting `failbit`. The stream layer is a raw serialization compatibility boundary, not an arithmetic-ready one.

Constructors, `set_str`, expression evaluation, and arithmetic-producing paths continue to reject zero denominators and produce canonical rationals. Before passing a stream-loaded value to GMP rational arithmetic, validate and canonicalize it with `mpq_class::canonicalize()` (which first rejects a zero denominator with `std::domain_error`, then calls `mpq_canonicalize`) or `gmpxx::mpq_canonicalize_checked(mpq_ptr)`. If the canonicalization throws, the target retains its original raw stream state and is not arithmetic-ready.

13 Move semantics and layout guarantees

Floating wrappers are nothrow move-constructible and add no per-object ownership flag around the backend object. Their size and alignment are part of the performance contract:

```
sizeof(gmpxx::mpf_class) == sizeof(mpf_t);
alignof(gmpxx::mpf_class) == alignof(mpf_t);
sizeof(mpfrxx::mpfr_class) == sizeof(mpfr_t);
sizeof(mpfrxx::mpc_class) == sizeof(mpc_t);
```

This preserves dense array stride for BLAS-like kernels: an array of `mpfr_class` has the same stride as an array of `mpfr_t`, with no extra padding from a wrapper-side validity flag.

Move construction keeps the moved-from source as a valid backend object by initializing destination storage and swapping; this may allocate but avoids ownership metadata. In normal builds, move assignment to a valid destination preserves the destination precision. Under GMPFRXX_MKII_FAST_FIXED_PREC, move assignment may assume that source and destination precisions match and take the fast swap path; a mismatch is a caller contract violation.

A moved-from floating object remains valid for destruction, assignment, and swap, but its value and precision are unspecified until reassigned. Portable code must not read a moved-from

wrapper for numeric meaning or precision before assigning a new value. `mpfc_class` follows MPF move behavior through its two components.

14 Platform notes

Windows MPC division. On `_WIN32` builds, `mpc_class` division may use a scaled finite-value workaround instead of calling `mpc_div` directly. It is a portability guard for MinGW/MPC builds where a direct division can overflow near MPFR’s 32-bit-style exponent limits even when the exact quotient is finite and small. The workaround is narrow: it applies only when all components are finite, the divisor is not $0 + 0i$, and at least one component’s exponent is within a small margin of the current MPFR `emin/emax`. It performs componentwise max-scaling in MPFR temporaries and then rounds with the requested MPC rounding mode. It is a compatibility path, not a separate complex-division semantics; last-bit differences from a direct `mpc_div` are possible. Non-Windows builds call `mpc_div` directly.

15 Benchmark results

The repository contains reproducible benchmark reports under `benchmarks/gmp/*/README.md` and `benchmarks/mpfr/*/README.md`. Those reports are the authoritative source for raw logs, CSV files, plots, hotpath disassembly, exact command lines, and per-variant interpretation. This section records representative headline results from the committed benchmark data used for the 1.0.0 release-candidate documentation.

The 00/01/02 vector and matrix-vector rows use the repeat-10 runs recorded by `benchmarks/run_all.sh`. The Rgemm rows use the clean 512-bit pow2/64n sweep recorded by `benchmarks/run_rgemm_all.sh`. MFLOPS values are elapsed-time metrics, not hardware-counter measurements. In the table below, entries of the form “Max / Avg” report maximum and average MFLOPS over repeats. Rgemm rows show the representative large-size maximum at $N = 1472$, where most sizes were measured with `repeat=1`.

Backend	Benchmark	Case	Best serial representative	Best OpenMP representative
GMP	Rdot	512-bit	mkII_06_FIXED, 33.796 / 32.961	omp_mkII_03, 578.634 / 573.121
GMP	Rdot	1024-bit	mkII_06_FIXED, 12.151 / 11.897	omp_mkII_03, 323.366 / 315.184
GMP	Raxpy	512-bit	C_native_03, 34.040 / 33.727	omp_orig_02, 398.110 / 393.715
GMP	Raxpy	1024-bit	C_native_01, 12.042 / 11.970	omp_orig_02, 255.925 / 252.424
GMP	Rgemv	512-bit	mkII_03, 31.991 / 31.449	C_omp_07, 552.697 / 540.759
GMP	Rgemv	1024-bit	mkII_03_FIXED, 11.586 / 11.268	C_omp_07, 264.362 / 262.194
GMP	Rgemm	512-bit, =1472	N C_native_02, 31.346	C_omp_06, 871.995
MPFR	Rdot	512-bit	C_native_06, 23.936 / 23.688	C_omp_01_FMA, 600.518 / 560.417
MPFR	Rdot	1024-bit	mkII_05_RPF, 10.425 / 10.183	C_omp_01_FMA, 293.591 / 286.659
MPFR	Raxpy	512-bit	mkII_01_RPF, 23.209 / 22.963	C_omp_01_FMA, 422.804 / 415.811
MPFR	Raxpy	1024-bit	C_native_01_FMA, 9.393 / 9.351	omp_mkII_01_RPF, 256.399 / 252.310
MPFR	Rgemv	512-bit	mkII_02_ROUNDING_FMA, 23.959 / 23.408	omp_mkII_07_RP, 455.524 / 443.905
MPFR	Rgemv	1024-bit	mkII_04_RPF, 10.434 / 10.152	C_omp_07_FMA, 239.287 / 233.270
MPFR	Rgemm	512-bit, =1472	N C_native_02, 24.069	C_omp_02, 686.758

Table entries are shortened for page fit. Prefix `omp_` denotes an OpenMP target, and `C_` denotes a raw C native target. The shortened suffix `FIXED` denotes the fixed-precision fastpath build (`FIXED_PRECISION_FASTPATH`). The shortened MPFR suffix `RPF` denotes the `ROUNDING_PRECISION_FMA` class. Full executable names, sorted tables, raw-data paths, plots, and disassembly snippets are intentionally kept in the benchmark README files rather than duplicated here.

Interpretation. The committed data show three stable boundaries. First, reusable temporaries and fixed-precision scratch paths control most GMP wrapper overhead in expression benchmarks. Second, MPFR wrappers need loop-external rounding plus an FMA-capable source shape before their hot loop reaches the raw C FMA call class. Third, Rgemm is not primarily a wrapper problem in the current sweep: at N=1472, the best GMP and MPFR wrappers are close to their raw C backend class, while GMP remains about 1.3 times faster than MPFR in the best serial C native kernels.

The benchmark reports should be consulted before drawing narrower conclusions. They include static sorted tables, line plots, and representative disassembly showing whether `mpf_init2` / `mpfr_init2`, rounding-mode lookup, temporary construction, `mpfr_fma`, and OpenMP final reductions appear in or outside the timed hot loop.

16 Version information

```
const char* gmpxx::version();           // also mpfrxx::version()
const char* gmpxx::git_commit_hash();   // also mpfrxx::git_commit_hash()
std::ostream& gmpxx::print_version(std::ostream&);
std::ostream& gmpxx::print_git_commit_hash(std::ostream&);
```

17 License

gmpfrxx_mkII is distributed under the BSD 2-Clause license, Copyright © 2026 Nakata, Maho. The full text appears in the header of each source file.